02. Fast Path 컨센서스 동작과정

목차

1. Fast Path 개념
2. 라운드 관리
3. 인증서 처리 과정
4. 타임아웃 처리
5. 조기 커밋 메커니즘
6. 성능 최적화

Fast Path 개념

Fast Path는 정상적인 네트워크 조건에서 빠른 컨센서스를 달성하기 위한 최적화된 경로입니다. 비잔틴 노드가 f개일 때, **2f+1개의 노드**만으로 합의를 이룰 수 있어 기존 3f+1 요구사항보다 빠른 처리가 가능합니다.

Fast Path 조건

```
impl FastPathConsensus {

pub fn new(committee: Committee, base_timeout: Duration) -> Result<Self, Autob

let total_nodes = committee.size();

let byzantine_threshold = (total_nodes - 1) / 3; // f = (n-1)/3

let fast_path_threshold = Stake::from((2 * byzantine_threshold + 1) as u64); //
```

```
Ok(Self {
committee,
fast_path_threshold,
byzantine_threshold,
// ...
})
}
}
```

라운드 관리

라운드 시작

```
impl FastPathConsensus {
pub fn start_round(&mut self, round: Round) -> Result<(), AutobahnError> {
// 이미 활성화된 라운드인지 확인
if self.active_rounds.contains_key(&round) {
return Err(AutobahnError::InvalidConfiguration {
details: format!("Round {} is already active", round.0),
});
}
// 새 라운드 정보 생성
let round_info = FastRoundInfo {
```

```
round,
votes: Vec::new(),
start_time: Timestamp::now(),
threshold_reached: false,
};
// 라운드 활성화
self.active_rounds.insert(round, round_info);
self.current_round = round;
self.round_start_time = Some(Instant::now());
Ok(())
}
}
```

라운드 정보 구조

```
pub struct FastRoundInfo {
pub round: Round,
pub votes: Vec<FastPathVote>,
pub start_time: Timestamp,
pub threshold_reached: bool,
}

pub struct FastPathVote {
```

```
pub round: Round,
pub voter: String,
pub signature: Vec<u8>,
pub timestamp: Timestamp,
}
```

인증서 처리 과정

1. 인증서 추가

```
impl FastPathConsensus {
pub fn add_certificate(&mut self, round: Round, certificate: Certificate) ->
// 인증서 유효성 검증
if certificate.round() != round {
return Err(AutobahnError::CertificateValidationFailed {
reason: format!("Certificate round {} does not match expected round {}",
certificate.round().0, round.0),
});
}
// 위원회 멤버십 확인
if !self.committee.authorities().contains_key(certificate.origin()) {
return Err(AutobahnError::CertificateValidationFailed {
reason: format!("Certificate origin {:?} not in committee", certificate.origin()
```

```
});
}
// 대기 중인 인증서에 추가
self.pending_certificates
.entry(round)
.or_insert_with(Vec::new)
.push(certificate);
Ok(())
}
}
```

2. Fast Commit 조건 확인

```
impl FastPathConsensus {
pub fn can_fast_commit(&self, round: Round) -> bool {
if let Some(certificates) = self.pending_certificates.get(&round) {
// 스테이크 계산
let total_stake: Stake = certificates
.iter()
.map(|cert| self.committee.stake(cert.origin()))
.sum();
// 2f+1 임계값 달성 여부 확인
total_stake >= self.fast_path_threshold
```

```
} else {
false
}
}
pub fn calculate_round_stake(&self, round: Round) -> Stake {
if let Some(certificates) = self.pending_certificates.get(&round) {
certificates
.iter()
.map(|cert| self.committee.stake(cert.origin()))
.sum()
} else {
Stake::from(0)
}
}
}
```

3. 임계값 상태 업데이트

```
impl FastPathConsensus {
pub fn update_threshold_status(&mut self, round: Round) -> Result<bool, Autob
if let Some(round_info) = self.active_rounds.get_mut(&round) {
let threshold_reached = self.is_threshold_reached(round);
round_info.threshold_reached = threshold_reached;
```

```
Ok(threshold_reached)
} else {
Err(AutobahnError::InvalidConfiguration {
details: format!("Round {} is not active", round.0),
})
}
}
}
```

타임아웃 처리

타임아웃 감지

```
impl FastPathConsensus {
pub fn is_round_timed_out(&self, round: Round) -> bool {
if let Some(start_time) = self.round_start_time {
if self.current_round == round {
return start_time.elapsed() > self.timeout_manager.current_timeout();
}
}
false
}
```

```
}
```

타임아웃 처리 액션

```
pub enum TimeoutAction {
RetryWithCurrentTimeout { round: Round },
IncreaseTimeoutAndRetry { new_timeout: Duration, round: Round },
FallbackToSlowPath { reason: String, round: Round },
}

impl FastPathConsensus {
pub fn handle_round_timeout(&mut self, round: Round) -> Result<TimeoutAction,
// 타임아웃 기록
self.timeout_manager.record_timeout();
self.performance_metrics.record_timeout(round);
let timeout_stats = self.timeout_manager.get_stats();
// 연속 타임아웃 횟수에 따른 액션 결정
if timeout_stats.consecutive_timeouts >= 3 {
// 3회 연속 타임아웃 → Slow Path로 전환
Ok(TimeoutAction::FallbackToSlowPath {
reason: "Consecutive timeouts exceeded threshold".to_string(),
round,
})
```

```
} else if timeout_stats.consecutive_timeouts >= 2 {
// 2회 연속 타임아웃 → 타임아웃 증가 후 재시도
Ok(TimeoutAction::IncreaseTimeoutAndRetry {
new_timeout: timeout_stats.current_timeout,
round,
})
} else {
// 1회 타임아웃 → 현재 타임아웃으로 재시도
Ok(TimeoutAction::RetryWithCurrentTimeout { round })
}
}
}
```

네트워크 지연시간 기록

```
impl FastPathConsensus {
pub fn record_network_latency(&mut self, latency: Duration) {
self.timeout_manager.record_latency(latency);
self.performance_metrics.record_latency(latency);
}
pub fn record_round_success(&mut self, round: Round) {
self.timeout_manager.record_success();
self.performance_metrics.record_success(round);
```

```
}

}
```

조기 커밋 메커니즘

Fast Path 투표 추가

```
impl FastPathConsensus {

pub  fn  add_fast_path_vote(&mut  self, vote: FastPathVote) -> Result<bool, Auto

let  round = vote.round;

// 활성 라운드 확인

if !self.active_rounds.contains_key(&round) {

return Err(AutobahnError::InvalidConfiguration {

details: format!("Cannot vote for inactive round {}", round.0),

});

}

let  round_info = self.active_rounds.get_mut(&round).unwrap();

// 중복 투표 확인

if  round_info.votes.iter().any(|v| v.voter == vote.voter) {

return Err(AutobahnError::CertificateValidationFailed {

reason: format!("Duplicate vote from voter in round {}", round.0),

});

}
```

```
// 투표 추가
round_info.votes.push(vote);
// 임계값 달성 확인 (2f+1)
let vote_count = round_info.votes.len();
let threshold_reached = vote_count >= (2 * self.byzantine_threshold + 1);
if threshold_reached && !round_info.threshold_reached {
round_info.threshold_reached = true;
return Ok(true); // 조기 커밋 감지
}
Ok(false)
}
}
```

조기 커밋 증명 생성

```
impl FastPathConsensus {
pub fn generate_early_commit_proof(&self, round: Round) -> Result<EarlyCommitP
let round_info = self.active_rounds.get(&round)
.ok_or_else(|| AutobahnError::InvalidConfiguration {
details: format!("Round {} is not active", round.0),
})?;
if !round_info.threshold_reached {
```

```
return Err(AutobahnError::InvalidConfiguration {
details: format!("Round {} has not reached threshold", round.0),
});
}
// 투표에서 서명과 참여자 수집
let mut aggregated_signatures = Vec::new();
let mut participants = Vec::new();
for vote in &round_info.votes {
aggregated_signatures.extend_from_slice(&vote.signature);
participants.push(vote.voter.clone());
}
Ok(EarlyCommitProof {
round,
aggregated_signatures,
participants,
})
}
}
```

Fast Path 인증서 생성

```
impl FastPathConsensus {
pub fn create_fast_path_certificate(
```

```
&self,
round: Round,
base_certificate: Certificate
) -> Result<AutobahnCertificate, AutobahnError> {
let round_info = self.active_rounds.get(&round)
.ok_or_else(|| AutobahnError::InvalidConfiguration {
details: format!("Round {} is not active", round.0),
})?;
// 조기 커밋 증명 생성
let early_commit_proof = self.generate_early_commit_proof(round)?;
// Fast Path 정보 생성
let fast_path_info = FastPathInfo {
fast_commit_round: round,
fast_path_votes: round_info.votes.clone(),
early_commit_proof: Some(early_commit_proof),
};
// 타임아웃 정보 생성
let timeout_stats = self.timeout_manager.get_stats();
let timeout_info = TimeoutInfo {
current_timeout: timeout_stats.current_timeout,
base_timeout: timeout_stats.base_timeout,
network_latency: timeout_stats.avg_latency,
};
```

```
// Autobahn 인증서 생성
let autobahn_cert = AutobahnCertificate::with_fast_path_info(
base_certificate,
fast_path_info,
timeout_info,
);
Ok(autobahn_cert)
}
}
```

성능 최적화

Fast Path 실행 가능성 평가

```
pub enum FastPathViability {
Good { confidence: f64 },
Degraded { reason: String, confidence: f64 },
ShouldFallback { reason: String, confidence: f64 },
}

impl FastPathConsensus {
pub fn evaluate_fast_path_viability(&self) -> FastPathViability {
let metrics = &self.performance_metrics;
```

```
let timeout_stats = self.timeout_manager.get_stats();
// 성공률 계산
let total_rounds = metrics.total_rounds_attempted;
let success_rate = if total_rounds > 0 {
metrics.successful_rounds as f64 / total_rounds as f64
} else {
1.0 // 초기에는 양호하다고 가정
};
// 다양한 요소를 기반으로 평가
if timeout_stats.consecutive_timeouts >= 3 {
FastPathViability::ShouldFallback {
reason: "Too many consecutive timeouts".to_string(),
confidence: 0.9,
}
} else if success_rate < 0.5 && total_rounds >= 5 {
FastPathViability::ShouldFallback {
reason: "Low success rate".to_string(),
confidence: 0.8,
}
} else if metrics.average_latency > timeout_stats.current_timeout / 2 {
FastPathViability::Degraded {
reason: "High network latency".to_string(),
confidence: 0.7,
```

```
}
} else {
FastPathViability::Good {
confidence: success_rate,
}
}
}
}
```

성능 메트릭 추적

```
pub struct FastPathPerformanceMetrics {
pub total_rounds_attempted: u64,
pub successful_rounds: u64,
pub timed_out_rounds: u64,
pub average_latency: Duration,
pub recent_latencies: VecDeque<Duration>,
}

impl FastPathPerformanceMetrics {
pub fn record_success(&mut self, _round: Round) {
self.successful_rounds += 1;
self.total_rounds_attempted += 1;
```

```
self.last_updated = Instant::now();
}
pub fn record_timeout(&mut self, _round: Round) {
self.timed_out_rounds += 1;
self.total_rounds_attempted += 1;
self.last_updated = Instant::now();
}
pub fn record_latency(&mut self, latency: Duration) {
if self.recent_latencies.len() >= 20 {
self.recent_latencies.pop_front();
}
self.recent_latencies.push_back(latency);
// 평균 지연시간 업데이트
if !self.recent_latencies.is_empty() {
let total: Duration = self.recent_latencies.iter().sum();
self.average_latency = total / self.recent_latencies.len() as u32;
}
}
}
```

라운드 정리 및 메모리 관리

```
impl FastPathConsensus {
```

```
pub fn cleanup_old_certificates(&mut self, keep_rounds: usize) {
let cutoff_round = if self.current_round.0 > keep_rounds as u64 {
Round(self.current_round.0 - keep_rounds as u64)
} else {
Round(0)
};
// 오래된 대기 인증서 정리
self.pending_certificates.retain(|&round, _| round >= cutoff_round);
// 오래된 활성 라운드 정리
self.active_rounds.retain(|&round, _| round >= cutoff_round);
}
pub fn finalize_round(&mut self, round: Round) -> Result<Option<FastRoundInfo
if let Some(round_info) = self.active_rounds.remove(&round) {
// 임계값에 도달한 경우 대기 인증서 정리
if round_info.threshold_reached {
self.pending_certificates.remove(&round);
}
Ok(Some(round_info))
} else {
Ok(None)
}
}
```

```
}
```

Fast Path 처리 흐름 요약

1. **라운드 시작**: `start_round()` 호출로 새 라운드 활성화
2. **인증서 수집**: `add_certificate()` 로 인증서 추가 및 검증
3. **임계값 확인**: `can_fast_commit()` 으로 2f+1 스테이크 달성 여부 확인
4. **조기 커밋**: 임계값 달성 시 `create_fast_path_certificate()` 호출
5. **타임아웃 처리**: 시간 초과 시 `handle_round_timeout()` 으로 적절한 액션 결정
6. **성능 추적**: 모든 과정에서 메트릭 수집 및 분석

Fast Path는 정상적인 네트워크 조건에서 빠른 컨센서스를 제공하며, 문제 발생 시 자동으로 Slow Path로 전환하여 안정성을 보장합니다.